JAVA WEB PROGRAMMING

Data Validation in Spring Boot Applications

Bean Validation · Thymeleaf · REST API · Spring Boot 4.0.5

Algebra Bernays University 2025/2026

Agenda

What we will cover today

01 **Why Validate?**

Importance & validation approaches overview

02 **Validation Annotations**

@NotEmpty, @Size, @Email, @Pattern, @Min/@Max, @PositiveOrZero, @Phone…

03 **Where to Validate?**

Command Object vs Domain Object vs DTO (MVC vs REST API)

04 **@Valid & BindingResult**

The mechanics of triggering and capturing validation errors

05 **Error Messages**

Customising messages, ValidationMessages.properties

06 **Displaying Errors**

Thymeleaf field errors, global errors, REST API error responses

07 **Global vs Local Messages**

Field-level vs cross-field business rule validation

08 **Demo Project**

Running example: Thymeleaf MVC + REST API (Spring Boot 4.0.5)

Why Validate Data?

Security, data integrity and user experience

Security

Prevent SQL injection, XSS, and malformed data attacks. Untrusted input is a primary attack vector.

Data Integrity

Ensure only correct, consistent data reaches the database. Avoid null pointer exceptions and corrupt records.

User Experience

Provide immediate, clear feedback so users can correct mistakes without losing entered data.

Business Rules

Enforce application-level constraints like age limits, phone format per country, or unique email addresses.

Spring Boot uses the Bean Validation API (Jakarta Validation) — annotations describe constraints, the framework enforces them automatically.

Validation Annotations

Jakarta Bean Validation 3.x — key annotations at a glance

Annotation	Applies To	Description
@NotNull	*Any*	Value must not be null
@NotEmpty	*String / Collection / Map / Array*	Must not be null and must have size > 0
@NotBlank	*String*	Must not be null and must contain at least one non-whitespace character
@Size(min, max)	*String / Collection / Map / Array*	Size/length must be between min and max (inclusive)
@Min(value)	*Number*	Value must be ≥ specified minimum
@Max(value)	*Number*	Value must be ≤ specified maximum
@Email	*String*	Must be a well-formed email address
@Pattern(regexp)	*String*	Must match the given regular expression
@PositiveOrZero	*Number*	Value must be ≥ 0 (positive or zero)
@Positive / @Negative	*Number*	Value must be strictly positive / strictly negative
@Past / @Future	*Date / Time*	Must be a past / future date-time

Validation Annotations — Code Example

Annotating a domain class

```
public class UserDto {
    @NotNull  @Min(value = 1)
    private Long id;
    @NotBlank(message = "Name is required")
    @Size(min = 2, max = 100)
    private String fullName;
    @NotBlank  @Email(message = "Invalid email format")
    private String email;
    @NotNull  @Min(value = 18, message = "Must be at least 18")
    @Max(value = 120)
    private Integer age;
    @Pattern(regexp = "^\\+?[0-9\\s\\-]{7,15}$", message = "Invalid phone")
    private String phoneNumber;
    @PositiveOrZero  // >= 0
    private Integer loyaltyPoints;
}
```

@NotBlank

String only — rejects null, empty AND blank (spaces only)

@NotEmpty

String / Collection — rejects null and empty (but allows blank spaces)

@Email

Validates e-mail address format using RFC 5322

@Pattern

Custom regex — use for phone, postal code, custom formats

@PositiveOrZero

Number must be ≥ 0; use @Positive for strictly > 0

Where to Put Validation Annotations?

Command Object · Domain Object · DTO (REST API)

Command Object

(MVC — Form Backing Object)

✓ Pros

- Keeps validation separate from domain
- Reusable for different forms
- Spring MVC binds form fields automatically

✗ Cons

- Extra class per form
- Manual mapping to domain object

```
// Dedicated class for the form
public class UserForm {
  @NotBlank
  private String fullName;
  @Email
  private String email;
}
```

Domain Object

(Entity / Model class directly)

✓ Pros

- Simple — single class
- No extra mapping code
- Common in small apps

✗ Cons

- Mixes persistence & validation concerns
- Same constraints for all use-cases

```
@Entity
public class User {
  @Id @GeneratedValue
  private Long id;
  @NotBlank @Email
  private String email;
}
```

DTO

(Data Transfer Object — REST API)

✓ Pros

- Clean API contract
- Decoupled from persistence layer
- Different DTOs per endpoint (create/update)

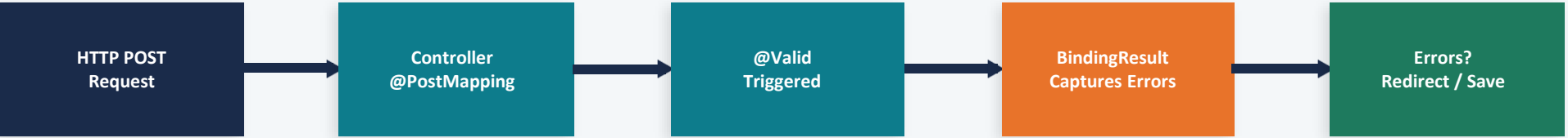
✗ Cons

- More classes to maintain
- Mapping layer required (MapStruct / ModelMapper)

```
// Used as @RequestBody
public class UserRequest {
  @NotBlank @Email
  private String email;
  @NotNull @Min(18)
  private Integer age;
}
```

@Valid Annotation & BindingResult

Triggering validation and capturing errors in the Controller



@Valid

Place BEFORE the method parameter to trigger Bean Validation. Spring invokes the validator automatically.

BindingResult

Must immediately follow the validated parameter. Holds FieldErrors and ObjectErrors. result.hasErrors() returns true if any constraint was violated.

@Validated

Spring-specific alternative to @Valid. Supports validation groups — validate different subsets of constraints per use-case.

```
@PostMapping("/users/add")
public String addUser(@Valid UserForm form,
                      BindingResult result,
                      Model model) {
    if (result.hasErrors()) {
        // Stay on form, Thymeleaf shows errors
        return "users/addForm";
    }
    userService.save(form);
    return "redirect:/users";
}
```

Customising Error Messages

Inline, properties file, and message interpolation

Three Ways to Define Error Messages:

1 Inline — message attribute directly on the annotation

```
@NotBlank(message = "Full name is required and cannot be empty.")  private String fullName;
```

2 ValidationMessages.properties — externalise and internationalise messages

```
# src/main/resources/ValidationMessages.properties
user.name.required=Full name is required.
user.email.invalid=Please enter a valid email address.
```

```
// Reference from annotation:
@NotBlank(message = "{user.name.required}")
@Email(message = "{user.email.invalid}")
```

3 Spring MVC message interpolation — use field names in messages.properties

```
# messages.properties (Spring MessageSource)
NotBlank.userForm.fullName=Name required.
# Pattern: {AnnotationName}.{objectName}.{fieldName}
```

Spring resolves messages in order:

1. {Annotation}.{objectName}.{field}
2. {Annotation}.{field}
3. {Annotation}.{fieldType}
4. {Annotation} (default)

Displaying Validation Errors — Thymeleaf

Field errors, errorclass, th:errors and th:each

Method 1 — th:errors (inline, single field)

```
<div>
  <label>Email</label>
  <input th:field="*{email}" />
  <p th:if="${#fields.hasErrors('email')}"
     th:errorclass="error"
     th:errors="*{email}"></p>
</div>
```

Method 2 — th:each (iterate per-field errors)

```
<ul>
  <li th:each="err : ${#fields.errors('fullName')}"
      th:text="${err}" class="error"></li>
</ul>
```

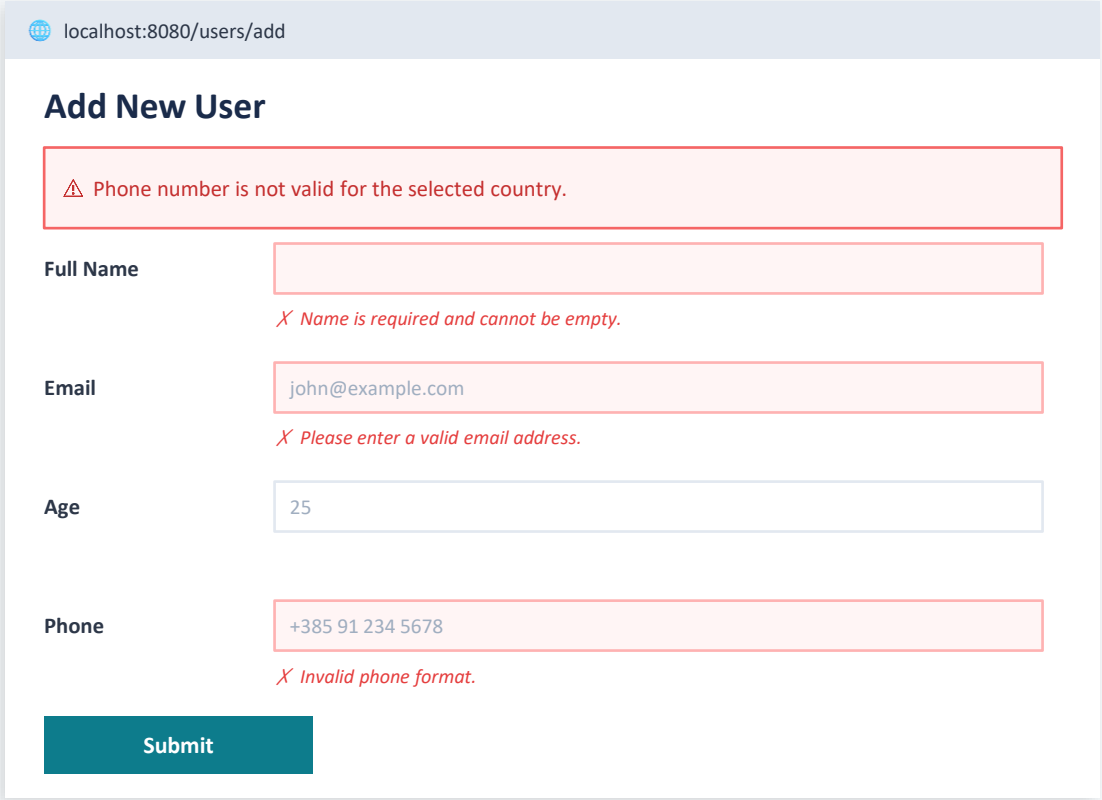
Method 3 — All errors at once (global summary)

```
<div th:if="${#fields.hasAnyErrors()}">
  <ul>
    <li th:each="err : ${#fields.allErrors()}"
        th:text="${err}"></li>
  </ul>
</div>
```

Note: Use th:errorclass="error" to automatically add a CSS class when a field has errors. Define .error { color: red; border: 1px solid red; } in your stylesheet.

Wireframe — Thymeleaf Form with Validation Errors

How field-level and global errors are rendered in the browser



Annotations Producing These Errors:

- Field-level error — shown directly below the invalid input
- Global error — shown at the top of the form (business rule)
- Error CSS class applied via th:errorclass to input border
- Valid field — no error class, normal border

Thymeleaf Template Pattern:

```
<p th:if=
"${#fields.hasErrors('name')}"
   th:errors="*{name}"
   th:errorclass="error">
</p>
```

Global vs Local (Field-Level) Validation Messages

Understanding the difference with practical examples

Local (Field-Level) Messages

Attached to a specific field on the object. The violation is associated with a named property.

Example:

Email field is empty, name too short, age out of range.

Thymeleaf:

```
th:errors="*{email}"  or
#fields.errors('email')
```

REST API response:

```
"field": "email",
"message": "Invalid email"
```

Global (Object-Level) Messages

Not attached to a single field. Represent cross-field or business rules involving multiple fields.

Example:

Phone must start with country code. Password and confirm password must match.

Thymeleaf:

```
#fields.hasErrors('global')
#fields.errors('global')
```

REST API response:

```
"field": null,
"message": "Phone invalid for country"
```

Adding & Displaying Global Errors — Code

ObjectError, result.addError() and Thymeleaf global display

1 Validation Service (cross-field business rule)

```
@Service
public class
UserValidationService {
    public Optional<String> validateCrossField(UserForm f) {
        if (f.getCountry() != null && f.getPhone() != null && f.getCountry().equals("HR") && !f.getPhone().startsWith("385")) {
            return Optional.of("Phone must start with 385 for Croatia.");
        }
        return Optional.empty();
    }
}
```

2 Controller — inject global error into BindingResult

```
validationService.validateCrossField(form)
  .ifPresent(msg -> {

      result.addError(new ObjectError(
          "userForm", msg
      ));
});
```

3 Thymeleaf — display global errors

```
<div th:if="${#fields.hasErrors('global')}">
  <p>Global Errors:</p>
  <p th:each="e:${#fields.errors('global')}"
     th:text="${e}" class="error">
  </p>
</div>
```

⚠ Important: BindingResult must immediately follow the @Valid parameter in the method signature. If you place another parameter between them, Spring will throw an exception before validation completes.

Validation in REST API Applications

@RequestBody validation and structured error responses

Key Differences: MVC vs REST API Validation Handling

Aspect	MVC (Thymeleaf)	REST API (@RestController)
Object type	Command Object / Domain Object	DTO (Data Transfer Object)
Annotation on param	@Valid @ModelAttribute UserForm	@Valid @RequestBody UserRequest
Error handling	BindingResult + return view name	@ExceptionHandler(MethodArgumentNotValidException)
Error format	Thymeleaf template re-renders with errors	JSON body with list of field errors
BindingResult needed?	Yes — must follow @Valid param	No — exception handler is preferred pattern

REST Controller — @ExceptionHandler approach

```
@PostMapping("/api/users")
public ResponseEntity<UserDto> create(
    @Valid @RequestBody UserRequest req) {
    // If invalid, exception is thrown
    // and handled by @ExceptionHandler
    return ResponseEntity.ok(service.create(req));
}
```

Global Exception Handler

```
@RestControllerAdvice
public class ValidationExceptionHandler {
  @ExceptionHandler(MethodArgumentNotValidException.class)
  public ResponseEntity<List<ErrorDto>> handle(
          MethodArgumentNotValidException ex) {
    // extract & return errors
  }
}
```

Wireframe — REST API Validation Error Response

JSON error payload returned to the API client

HTTP Request (invalid data sent)

```
POST /api/users  HTTP/1.1
Content-Type: application/json

{
  "fullName": "",
  "email": "not-an-email",
  "age": -5,
  "phoneNumber": "abc123"
}
```

400 ▶
Bad Request

HTTP 400 Response (structured JSON errors)

```
{
  "status": 400,
  "errors": [
    { "field": "fullName",
      "message": "Name required" },
    { "field": "email",
      "message": "Invalid email" },
    { "field": "age",
      "message": "Age must be >= 18" }
  ]
}
```

ErrorDto — response body structure

```
public record ErrorDto(
    String field,   // null = global error
    String message
) {}

// In handler:
ex.getBindingResult()
  .getFieldErrors()
  .stream()
  .map(e -> new ErrorDto(e.getField(), e.getDefaultMessage()))
  .toList()
```

Exception Handler Mapping

```
@ExceptionHandler
  (MethodArgumentNotValidException.class)
public ResponseEntity<?> handle(
        MethodArgumentNotValidException ex) {
  var errors = /* map errors */;
  return ResponseEntity.badRequest().body(errors);
}
```

Demo Project — Structure & Setup

Spring Boot 4.0.5 — Thymeleaf MVC + REST API in one project

Project structure (src/main/java + resources):

```
validation-demo/
├── pom.xml
└── src/main/
    ├── java/.../
    │   ├── ValidationDemoApp.java
    │   ├── domain/
    │   │   └── User.java
    │   ├── dto/
    │   │   ├── UserForm.java   (MVC)
    │   │   └── UserRequest.java   (REST)
    │   ├── controller/
    │   │   ├── UserMvcController.java
    │   │   └── UserRestController.java
    │   ├── service/
    │   │   └── UserValidationService.java
    │   └── exception/
    │       └── GlobalExceptionHandler.java
    └── resources/
        ├── application.properties
        ├── ValidationMessages.properties
        ├── messages.properties
        └── templates/
            ├── addUser.html
            └── users.html
```

Key pom.xml dependencies:

```
<parent>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>4.0.5</version>
</parent>

<!-- Thymeleaf + Spring MVC -->
<dependency>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>

<!-- Bean Validation -->
<dependency>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>

<!-- Spring Data JPA (H2 demo) -->
<dependency>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <artifactId>h2</artifactId>  <scope>runtime</scope>
</dependency>
```

Summary & Key Takeaways

What you should know after this lecture

1

Annotations Declare Constraints

Use @NotBlank, @Email, @Size, @Min/@Max, @Pattern, @PositiveOrZero to annotate your DTO or command object fields.

2

Choose the Right Class

Command Object for MVC forms, DTO for REST API. Keep domain entities free of UI-specific validation where possible.

3

@Valid Triggers Validation

Place @Valid before the parameter and BindingResult immediately after in MVC. For REST, let @ExceptionHandler handle MethodArgumentNotValidException.

4

Customise Messages

Inline message=, ValidationMessages.properties, or Spring's messages.properties. Use key interpolation like {user.email.invalid} for i18n.

5

Field vs Global Messages

Field errors attach to a specific property. Global (ObjectErrors) cover cross-field business rules — add via result.addError(new ObjectError(...)).

6

Thymeleaf vs REST Display

Thymeleaf: th:errors, th:each on #fields. REST: return JSON list of ErrorDto from @ExceptionHandler with HTTP 400.

References

Spring Boot Reference Docs — https://docs.spring.io/spring-boot/reference/

Jakarta Bean Validation Spec — https://jakarta.ee/specifications/bean-validation/3.0/

Baeldung — Thymeleaf Error Messages — https://www.baeldung.com/spring-thymeleaf-error-messages

Baeldung — Validation in Spring Boot — https://www.baeldung.com/spring-boot-bean-validation

Spring MVC Validation Guide — https://www.baeldung.com/spring-mvc-custom-validator

Hibernate Validator (Reference Implementation) — https://hibernate.org/validator/documentation/

Thank you for your attention!

Demo project available in the course repository. Open in IntelliJ IDEA and run ValidationDemoApp.java